

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 1.6

Vibe-Code a React Chat UI Skeleton

Module	1 — LLM Foundations & Vibe Coding with Claude
Topic	1.6 — a reusable chat widget shell, built by vibe coding
Estimated time	30–40 minutes
Tools needed	Terminal, Claude Code CLI, Node.js 20+, a browser
Prerequisite	Topics 1.3 and 1.4 on the program website; Lab 1.5 completed

What You're Building

GOAL

A React functional component — `ChatWidget` — with a scrollable message list, a controlled text input, and a "send" handler. No backend, no API calls yet: local state only.

This is UI-only, but it isn't throwaway: the state shape you build here is the exact shape a real agent backend gets wired into later in the program, so get it right now rather than fixing it later.

Steps 1–2: Scaffold the Project

1 Create a React app with Vite

TERMINAL

```
npm create vite@latest chat-widget -- --template react
cd chat-widget
npm install
```

What you'll see: a new `chat-widget` folder with a working, minimal React app scaffold — you haven't written any custom code yet.

2 Confirm the scaffold runs before changing anything

TERMINAL

```
npm run dev
```

What you'll see: a local dev server URL (usually `http://localhost:5173`). Open it in a browser — you should see Vite's default starter page. Leave this running; Vite hot-reloads as you edit.

Steps 3–5: Vibe-Code the Component

3 Open Claude Code and describe the component

In a second terminal tab (leave `npm run dev` running), inside the `chat-widget` folder, run `claude` and give it this spec:

PROMPT TO CLAUDE CODE

```
Build a React functional component called ChatWidget in src/ChatWidget.jsx.  
State: messages (array of {role, content} objects, role is "user" or  
"assistant"), and inputValue (string).  
Seed messages with one initial assistant greeting: "Hi! Ask me about  
our products."  
Render the message list scrollable, with user and assistant messages  
visually distinguished (e.g. alignment or background color).  
A controlled text input plus a send button appends a new  
{role: "user", content: inputValue} message to the list and clears  
the input. Pressing Enter should also send.  
No API calls – local state only for now. Use plain CSS in a  
ChatWidget.css file, no UI library. Wire ChatWidget into App.jsx  
so it renders on page load.
```

4 Review the diff before accepting

Claude Code will typically create `ChatWidget.jsx`, `ChatWidget.css`, and modify `App.jsx`. Before accepting, check:

Check	Why it matters
Is <code>messages</code> an array of exactly <code>{role, content}</code> objects — not a different shape?	This exact shape gets reused when the widget is wired to a real agent later
Does each list item have a stable <code>key</code> prop (not just the array index, ideally)?	Avoids React key warnings and rendering bugs as messages grow
Does the input actually clear after sending?	A controlled input that doesn't reset the state value won't visually clear
Is there any hint of a real network/fetch call?	There shouldn't be any yet — this lab is local state only

If the state shape doesn't match, don't hand-edit it — tell Claude Code directly: *"change messages to use {role, content} objects instead"* and let it produce the diff.

5 Try it in the browser

With `npm run dev` still running, refresh `http://localhost:5173`.

What you'll see: the initial assistant greeting message, an input box, and a send button. Type something and send it — it should appear in the list, and the input should clear.

Step 6: Verify Against the Checklist

6 Confirm all four acceptance criteria

- ☐ Typing and submitting a message appends it to the visible list
- ☐ The input clears after send
- ☐ The component renders sensibly with zero messages (temporarily clear the seed message and reload to check)
- ☐ Message state uses the `{role, content}` shape, confirmed by reading the code — not just by how it looks on screen

Troubleshooting

Symptom	Likely cause	Try this
Console warning: "Each child in a list should have a unique key prop"	List items rendered without a key, or using an unstable one	Ask Claude Code to add a stable key — e.g. an incrementing id per message
Typing in the input does nothing / input feels "frozen"	Input isn't wired to <code>onChange</code> , or value is stale	Confirm the input has both <code>value={inputValue}</code> and <code>onChange</code> updating it
New messages don't appear after clicking send	State update mutates the array instead of creating a new one	Ask Claude Code to use <code>setMessages(prev => [...prev, newMessage])</code> instead of pushing directly
Page is blank / red error overlay in the browser	A JS error in the component	Read the overlay's error message and file/line — paste it back to Claude Code and ask it to fix that specific error
<code>npm run dev</code> shows a port already in use	A previous dev server is still running	Stop the earlier process, or let Vite pick the next free port automatically

Where This Goes Next

Commit this component to your own local git repository once it passes the checklist. Later in the program, you'll extend this exact file — replacing the local-only send handler with a real call to your Shopify shopping agent — rather than rebuilding the UI from scratch.